SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-26
Course Coordinator Name		Dr. Jagjit Singh Dhatterwal	
Instructor(s) Name		Dr. Jagjit Singh Dhatterwal	
Course Code	23CS201PE401	Course Title	Blockchain Engineering
Year/Sem	III/II	Regulation	R25
Date and Day of Assignment	02-02-2026	Time(s)	9:00AM to 11:00AM
Duration	2 Hours	Applicable to Batches	(23CSBTB19, 23CSBTB20 23CSBTB21, 23CSBTB22 23CSBTB23, 23CSBTB24 23CSBTB25, 23CSBTB26)
Assignment Number: 03/12			

Q. No.	Question	Expected Time to complete

1	Understanding Smart Contract Development and Unit Testing through the Truffle Framework
	
	Objective
	
	Implement a simple Solidity Smart Contract and perform UNIT Testing using Truffle to verify contract functionality.
	
	Problem Statement (Equivalent to Given Cipher Practical)
	
	Develop a basic Personal Portfolio Smart Contract that stores and retrieves user profile data, and validate its behavior using Truffle unit tests.
	
	Requirements
	
	Development Environment
	• Install Node.js
	• Install Truffle Framework
	• Install Ganache (Local Blockchain)
	• Use VS Code with:
	  ○ Solidity Extension
	  ○ JavaScript Extension	

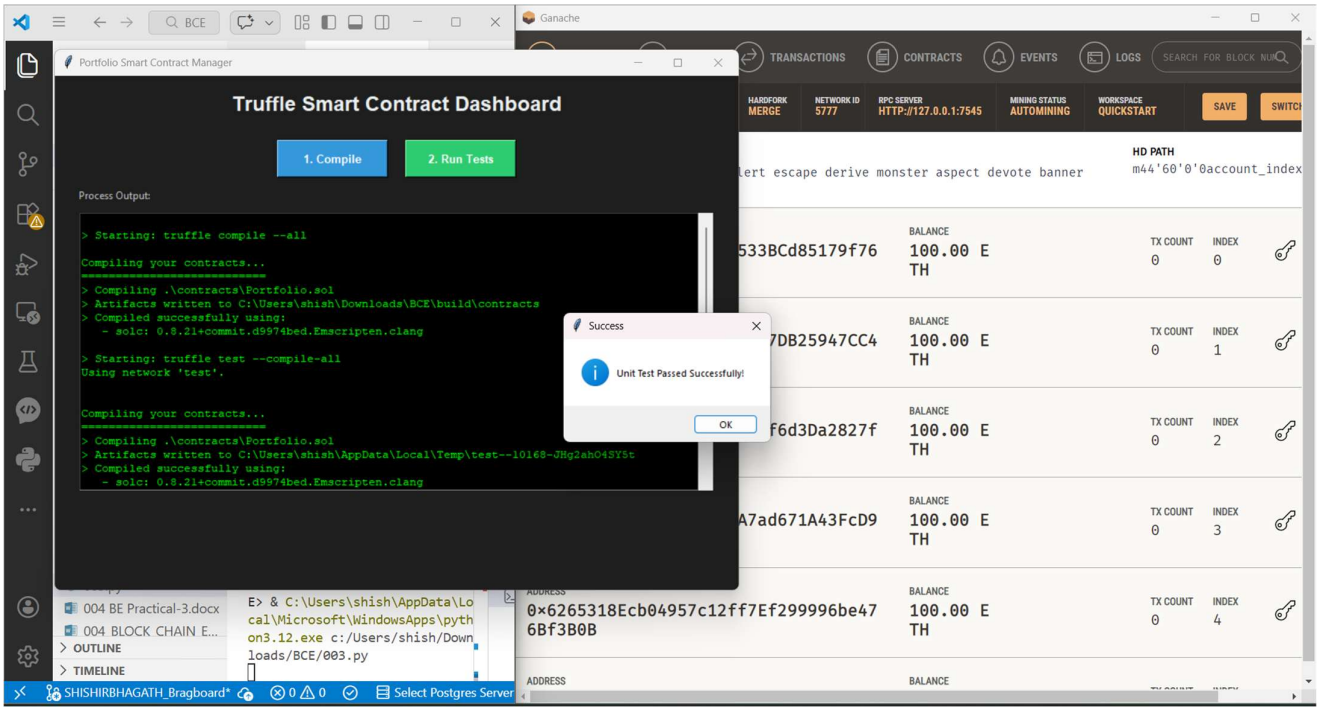
Code:

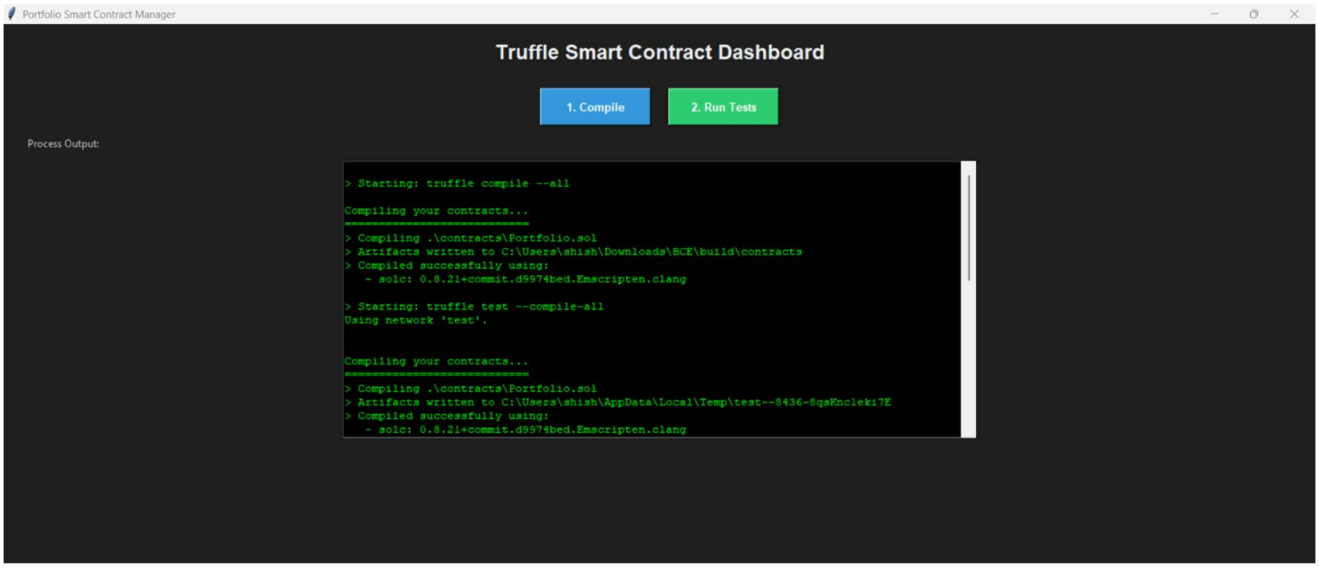
```python
import tkinter as tk
from tkinter import messagebox, scrolledtext
import subprocess
import os

class TruffleManager:
    def __init__(self, root):
        self.root = root
        self.root.title("Portfolio Smart Contract Manager")
        self.root.geometry("800x600")
        self.root.configure(bg="#1e1e1e")

        # Title
        tk.Label(root, text="Truffle Smart Contract Dashboard", fg="#ecf0f1",
                 bg="#1e1e1e", font=("Arial", 18, "bold")).pack(pady=15)

        # Control Buttons
        btn_frame = tk.Frame(root, bg="#1e1e1e")
        btn_frame.pack(pady=10)

        self.btn_compile = tk.Button(btn_frame, text="1. Compile", width=15, height=2,
                                     bg="#3498db", fg="white", font=("Arial", 10, "bold"),
                                     command=lambda: self.execute(["truffle", "compile", "--all"]))
        self.btn_compile.grid(row=0, column=0, padx=10)

        self.btn_test = tk.Button(btn_frame, text="2. Run Tests", width=15, height=2,
                                  bg="#2ecc71", fg="white", font=("Arial", 10, "bold"),
                                  command=lambda: self.execute(["truffle", "test", "--compile-all"]))
        self.btn_test.grid(row=0, column=1, padx=10)

        # Terminal Output
        tk.Label(root, text="Process Output:", fg="#bdc3c7", bg="#1e1e1e").pack(padx=25, anchor="w")
        self.terminal = scrolledtext.ScrolledText(root, width=90, height=20, bg="black", fg="#00ff00", font=("Courier New", 10))
        self.terminal.pack(pady=10, padx=20)

    def execute(self, command):
        self.terminal.insert(tk.END, f"\n> Starting: {' '.join(command)}\n")
        self.root.update_idletasks()

        try:
            # shell=True is vital for Windows to find truffle.cmd
            proc = subprocess.run(command, capture_output=True, text=True, shell=True)

            if proc.stdout:
                self.terminal.insert(tk.END, proc.stdout)
            if proc.stderr:
                # We show warnings (like uWS) but don't treat them as fatal
                self.terminal.insert(tk.END, f"\nSystem Warnings:\n{proc.stderr}")

            if "1 passing" in proc.stdout:
                messagebox.showinfo("Success", "Unit Test Passed Successfully!")

        except Exception as e:
            self.terminal.insert(tk.END, f"ERROR: {str(e)}")
            messagebox.showerror("Error", "Command failed to execute.")
```


```python
            if "1 passing" in proc.stdout:
                messagebox.showinfo("Success", "Unit Test Passed Successfully!")

        except Exception as e:
            self.terminal.insert(tk.END, f"ERROR: {str(e)}")
            messagebox.showerror("Error", "Command failed to execute.")
```


```python
        self.terminal.see(tk.END)

if __name__ == "__main__":
    root = tk.Tk()
    app = TruffleManager(root)
    root.mainloop()
```

Output:


Portfolio Smart Contract Manager
Truffle Smart Contract Dashboard
1. Compile
2. Run Tests
Process Output:
> Starting: truffle compile --all
Compiling your contracts...
> Compiling .\contracts\Portfolio.sol
> Artifacts written to C:\Users\shish\Downloads\BCE\build\contracts
> Compiled successfully using:
- solc: 0.8.21+commit.d9974bed.Emscripten.clang
> Starting: truffle test --compile-all
Using network 'test'.
Compiling your contracts...
> Compiling .\contracts\Portfolio.sol
> Artifacts written to C:\Users\shish\AppData\Local\Temp\test--10168-JHg2ahO4SY5t
> Compiled successfully using:
- solc: 0.8.21+commit.d9974bed.Emscripten.clang
Success
Unit Test Passed Successfully!
OK
Ganache
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
HD PATH
m44'60'0'0account_index
BALANCE
100.00 ETH
TX COUNT
INDEX
0x6265318Ecb04957c12ff7Ef299996be476Bf3B0B
004 BE Practical-3.docx
004 BLOCK CHAIN E...
OUTLINE
TIMELINE
E> & C:\Users\shish\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:/Users/shish/Downloads/BCE/003.py
SHISHIRBHAGATH_Bragboard*
Select Postgres Server


Portfolio Smart Contract Manager
Truffle Smart Contract Dashboard
1. Compile
2. Run Tests
Process Output:
> Starting: truffle compile --all
Compiling your contracts...
> Compiling .\contracts\Portfolio.sol
> Artifacts written to C:\Users\shish\Downloads\BCE\build\contracts
> Compiled successfully using:
- solc: 0.8.21+commit.d9974bed.Emscripten.clang
> Starting: truffle test --compile-all
Using network 'test'.
Compiling your contracts...
> Compiling .\contracts\Portfolio.sol
> Artifacts written to C:\Users\shish\AppData\Local\Temp\test--8436-8qsKncleki7E
> Compiled successfully using:
- solc: 0.8.21+commit.d9974bed.Emscripten.clang